Matriz de trazabilidad de requerimientos funcionales

Requerimiento	Historia de Usuario	Diseño / C4	API / OpenAPI	Implementación	BDD	Prueba	Estado
REQ-FN-001 – Autenticar usuario	HU-01	Módulo de Autenticación	POST /api/v1/auth/login	Auth Controller + JWT + PostgreSQL	autenticacion.feature	CP-01 / CP-02	Implementado
REQ-FN-002 – Registrar contacto	HU-02	Módulo Contactos / Service / Repository	POST /api/v1/contactos	ContactoController@store + PostgreSQL + invalidación Redis	gestion_recursos.feature	CP-04 / CP-05	Implementado
REQ-FN-003 – Consultar contactos	HU-03	Contactos / Repository / Redis	GET /api/v1/contactos	ContactoController@index + Cache-Aside	gestion_recursos.feature	CP-03 / CP-08	Implementado
REQ-FN-003 – Consultar detalle de contacto	HU-03	Contactos / Service / Repository	GET /api/v1/contactos/{id}	ContactoController@show + PostgreSQL + Redis	gestion_recursos.feature	Prueba funcional Postman	Implementado
REQ-FN-004 – Actualizar contacto	HU-04	Módulo Contactos	PUT /api/v1/contactos/{id}	Definido en arquitectura/contrato	Escenario de actualización	Caso asociado	Contratado / pendiente de evidencia funcional
REQ-FN-005 – Desactivar contacto	HU-05	Módulo Contactos	DELETE /api/v1/contactos/{id}	Baja lógica definida	Escenario de desactivación	Caso asociado	Contratado / pendiente de evidencia funcional

Requerimiento	Historia de Usuario	Diseño / C4	API / OpenAPI	Implementación	BDD	Prueba	Estado
REQ-FN-006 – Clasificar contacto	HU-06	Contactos / Cliente / Proveedor / Empleado	POST / PUT /contactos	Modelo relacional y lógica de clasificación	Scenario Outline de tipos	CP-04 / CP-05	Parcial
REQ-FN-007 – Buscar y filtrar contactos	HU-07	Contactos / Repository	GET /api/v1/contactos?buscar=&tipo=&estado=	Filtros implementados en listado	Escenario de consulta	CP-03	Implementado
REQ-FN-008 – Consultar historial	HU-09*	Componente Historial / MongoDB	GET /api/v1/contactos/{id}/historial	Diseñado como persistencia documental	Escenario historial	CP-06	Arquitectónico / no implementado
REQ-FN-009 – Consultar relaciones	HU-10*	Componente Relaciones / Neo4j	GET /api/v1/contactos/{id}/relaciones	Diseñado como persistencia de grafos	Escenario relaciones	CP-07	Arquitectónico / no implementado

Matriz de requerimientos no funcionales

Requerimiento	Evidencia arquitectónica	Implementación / Evidencia	BDD / Prueba	Estado
REQ-NF-001 – Seguridad y autenticación	C4 + ADR + API protegida	JWT Bearer Authentication	Login válido/inválido, CP-01 y CP-02	☑
REQ-NF-002 – Validación de datos	DTOs y reglas de validación	Request/DTO de contactos	Scenario Outline de datos inválidos, CP-05	☑
REQ-NF-003 – Semántica HTTP	Contrato REST	Uso de 200, 201, 401, 404, etc. conforme RFC 9110	Postman / OpenAPI	☑
REQ-NF-004 – Errores estandarizados	API-First / RFC 9457	Respuestas application/problem+json, incluyendo 404 de contacto	Escenarios negativos	☑
REQ-NF-005 – Mantenibilidad	Monolito Modular, C4 y ADR-001	Controller → Service → Repository	Revisión arquitectónica	☑
REQ-NF-006 – Rendimiento mediante caché	ADR-003 / Redis Cache-Aside	Redis + Predis + TTL	MISS → HIT, CP-08	☑
REQ-NF-007 – Disponibilidad ante fallo de caché	Escenario ISO 25010	PostgreSQL como fuente principal y Redis como caché	CP-09	☑
REQ-NF-008 – Contrato API	API-First	docs/api/openapi.yaml OpenAPI 3.0.3	Validación del contrato	☑

Trazabilidad de los cuatro endpoints funcionales demostrados

Endpoint funcional	Requerimiento	Persistencia / Tecnología	Evidencia
POST /api/v1/auth/login	REQ-FN-001	PostgreSQL + JWT	Respuesta 200 y token
GET /api/v1/contactos	REQ-FN-003 / REQ-NF-006	PostgreSQL + Redis	X-Cache: MISS/HIT
POST /api/v1/contactos	REQ-FN-002	PostgreSQL + Redis	201 Created + invalidación de caché
GET /api/v1/contactos/{id}	REQ-FN-003	PostgreSQL + Redis	200, MISS/HIT y 404 Problem Details